

**PENSAMIENTO COMPUTACIONAL**

**Juan Esteban Campos Cardona**

**1091203999**

**Nicolas Santiago Guzman Chala**

**1126907455**

**Yulian Stiven Diaz Pulido**

**1091203910**



**Profesora:**

**Carolina Londoño Idarraga**

**Universidad del Quindío**

**Facultad de ingeniería**

**Ingeniería de sistemas y computación**

**Programación 2**

**Armenia, Mayo de 2026**

# 1. ¿Qué se solicita finalmente?

Se solicita el desarrollo de una plataforma de gestión de eventos y venta de entradas en Java, que permita la interacción entre usuarios y administradores según sus roles y permisos.

El sistema debe permitir:

- Exploración de eventos disponibles
- Gestión de recintos, zonas y asientos
- Realización de compras y pagos
- Inclusión de servicios adicionales en las compras
- Gestión de incidencias dentro del sistema
- Generación de reportes en distintos formatos
- Aplicación de patrones de diseño y principios de diseño orientado a objetos

La solución debe estructurarse bajo una arquitectura modular, escalable y mantenible, haciendo uso de patrones creacionales, estructurales y de comportamiento.

# 2. ¿Qué información es relevante y cómo se agrupa?

## Usuario

**La entidad Usuario almacena información como:**

- idUsuario (String)
- nombreCompleto (String)
- correo (String)

- telefono (String)
  - historialCompras (List<Compra>)
- 

## Evento

### La entidad Evento almacena:

- idEvento (String)
  - nombre (String)
  - categoria (String)
  - descripcion (String)
  - ciudad (String)
  - fechaHora (LocalDateTime)
  - estado (EstadoEvento)
  - recinto (Recinto)
- 

## Recinto

### Representa el lugar físico del evento:

- idRecinto (String)
  - nombre (String)
  - direccion (String)
  - ciudad (String)
  - capacidadTotal (int)
- 

## Zona

- idZona (String o int)
  - nombre (String)
  - capacidad (int)
  - precioBase (double)
-

## Asiento

- idAsiento (String o int)
  - fila (String)
  - numero (int)
  - estado (EstadoAsiento)
- 

## Compra

- idCompra (String o int)
  - usuario (Usuario)
  - entradas (List<Entrada>)
  - fechaCreacion (LocalDateTime)
  - total (double)
  - estado (EstadoCompra)
- 

## Entrada

- idEntrada (String o int)
  - zona (Zona)
  - asiento (Asiento)
  - precioFinal (double)
  - estado (EstadoEntrada)
- 

## Pago(implementado como strategy)

- La entidad Pago representa el registro de la transacción realizada dentro del sistema, mientras que el comportamiento del proceso de pago se implementa mediante el patrón Strategy, permitiendo diferentes métodos de pago sin modificar la lógica de la compra
- 

## Tarifa

- idTarifa (String o int)
  - tipo (String)
  - valorBase (double)
  - recargos (double)
- 

## **Incidencia**

- idIncidencia (String o int)
- tipo (String)
- descripcion (String)
- fecha (LocalDateTime)
- entidadAfectada (String)

# **3. ¿Qué funcionalidades se solicitan?**

## **Gestión de usuarios**

- **RF-001:** El sistema permite el registro de usuarios en la plataforma.
  - **RF-002:** El sistema permite el inicio de sesión de los usuarios.
  - **RF-003:** El sistema permite la modificación de la información del perfil del usuario.
  - **RF-004:** El sistema permite consultar el historial de compras del usuario.
- 

## **Gestión de eventos**

- **RF-005:** El sistema permite consultar eventos disponibles en la plataforma.
  - **RF-006:** El sistema permite visualizar el detalle de un evento.
  - **RF-007:** El sistema permite controlar el estado de los eventos (publicado, pausado, cancelado).
  - **RF-008:** El sistema permite gestionar la información de conciertos, obras de teatro o conferencias.
  - **RF-009:** El sistema permite consultar disponibilidad de zonas y asientos de un evento.
-

## Gestión de recinto

- **RF-010:** El sistema permite administrar la información de recintos donde se realizan los eventos.
  - **RF-011:** El sistema permite organizar la distribución física de un evento dentro del recinto.
  - **RF-012:** El sistema permite gestionar la capacidad y disponibilidad de los recintos.
- 

## Gestión de zonas

- **RF-013:** El sistema permite organizar las zonas dentro de un recinto.
  - **RF-014:** El sistema permite asignar precios base a cada zona.
  - **RF-015:** El sistema permite gestionar diferentes tipos de zonas y su configuración.
- 

## Gestión de asientos

- **RF-016:** El sistema permite gestionar la disponibilidad de los asientos.
  - **RF-017:** El sistema permite cambiar el estado de los asientos (disponible, reservado, vendido o bloqueado).
  - **RF-018:** El sistema evita la compra de asientos que no estén disponibles.
- 

## Gestión de compras

- **RF-019:** El sistema permite crear compras de entradas.
  - **RF-020:** El sistema permite modificar compras antes de realizar el pago.
  - **RF-021:** El sistema permite cancelar compras.
  - **RF-022:** El sistema permite calcular el costo total de la compra.
  - **RF-023:** El sistema permite realizar el seguimiento del estado de la compra.
- 

## Gestión de entradas

- **RF-024:** El sistema genera entradas a partir de una compra.
- **RF-025:** El sistema permite validar la información de las entradas.

- **RF-026:** El sistema permite gestionar el estado de las entradas (activa, usada, anulada).
- 

## Gestión de pagos

- **RF-027:** El sistema permite procesar pagos utilizando diferentes estrategias de pago.
  - **RF-028:** El sistema permite validar el resultado de las transacciones de pago.
  - **RF-029:** El sistema permite registrar el estado del pago dentro del sistema.
- 

## Gestión de tarifas

- **RF-030:** El sistema permite calcular el precio final de una compra.
  - **RF-031:** El sistema permite incluir servicios adicionales como VIP, parqueadero o merchandising.
  - **RF-032:** El sistema permite ajustar el valor de las entradas según la tarifa aplicada.
- 

## Gestión de incidencias

- **RF-033:** El sistema permite registrar incidencias dentro de la plataforma.
  - **RF-034:** El sistema permite gestionar errores de pago y fallos operativos.
  - **RF-035:** El sistema permite notificar eventos relevantes mediante el patrón Observer.
- 

## Reportes

- **RF-036:** El sistema permite generar reportes en formato PDF.
- **RF-037:** El sistema permite generar reportes en formato CSV.
- **RF-038:** El sistema permite exportar información de compras y eventos.

## 4. ¿Cómo se distribuyen las funcionalidades?

El sistema se organiza en capas y responsabilidades claramente definidas:

### Capa de presentación (UI - JavaFX)

- Interacción con el usuario
- Visualización de eventos, compras y reportes
- Captura de acciones del usuario

### Capa de servicios

- **UsuarioService**: gestión de usuarios
- **EventoService**: administración de eventos y estados
- **CompraService**: lógica de compras, pagos y cancelaciones
- **IncidenciaService**: registro y notificación de incidencias
- **ReporteService**: generación de reportes

### Capa de dominio (modelo)

- Usuario
- Evento
- Compra
- Entrada
- Recinto, Zona, Asiento
- Pago, Tarifa, Incidencia

### Capa de patrones

- Estrategias de pago (Strategy)
- Gestión de estados (State)
- Notificaciones (Observer)
- Decoración de compras (Decorator)
- Adaptación de reportes (Adapter)



## 5. ¿Qué debo hacer para probar las funcionalidades?

Para validar el correcto funcionamiento del sistema, se deben realizar pruebas funcionales simuladas mediante la ejecución de los diferentes casos de uso implementados en Java. Estas pruebas permiten verificar que cada módulo del sistema cumpla con los requisitos establecidos y que las reglas de negocio se ejecuten correctamente.

Las pruebas se realizan principalmente desde la capa de servicios, ejecutando los métodos correspondientes a cada funcionalidad del sistema y validando los cambios en el estado de las entidades. Asimismo, se verifica el comportamiento de los patrones de diseño implementados, tales como State, Strategy y Observer.

La validación se lleva a cabo mediante la ejecución de escenarios controlados, donde se ingresan datos de prueba y se observa el resultado obtenido en el sistema, ya sea a través de consola o de la interfaz gráfica.

---

### Casos de prueba principales

#### Gestión de usuarios

- Registro de usuario con datos válidos. Resultado esperado: usuario creado correctamente.
- Inicio de sesión con credenciales correctas. Resultado esperado: acceso permitido al sistema.

#### Gestión de eventos

- Creación de un evento con información completa. Resultado esperado: evento almacenado y disponible.
- Cambio de estado de un evento a cancelado. Resultado esperado: notificación a observadores.

#### Gestión de asientos

- Selección de asiento disponible. Resultado esperado: asiento cambia a estado reservado.
- Intento de selección de asiento vendido. Resultado esperado: operación rechazada.

## Gestión de compras

- Creación de una compra con entradas seleccionadas. Resultado esperado: compra creada correctamente.
- Cancelación de compra antes del pago. Resultado esperado: estado actualizado a cancelada.

## Pagos

- Procesamiento de pago con método válido. Resultado esperado: compra cambia a estado pagada.
- Intento de pago en compra cancelada. Resultado esperado: operación rechazada.

## Reportes

- Generación de reporte en PDF. Resultado esperado: archivo generado correctamente.
- Generación de reporte en CSV. Resultado esperado: archivo exportado correctamente.

## 6. ¿Qué puedo reutilizar?

En el desarrollo de la plataforma se han implementado distintos patrones de diseño con el objetivo de facilitar la organización del sistema, mejorar su mantenibilidad y permitir la reutilización de componentes.

En primer lugar, se utilizó el patrón **Singleton**, el cual permite asegurar una única instancia encargada de gestionar la información global de la aplicación, centralizando el acceso a los datos del sistema.

Para la creación de eventos se aplicó el patrón **Factory Method**, el cual permite generar distintos tipos de eventos de manera estructurada y consistente.

Adicionalmente, el patrón **Prototype** se emplea para la creación de objetos a partir de la clonación de estructuras ya existentes, permitiendo reutilizar configuraciones base y reducir la complejidad en la generación de nuevos elementos como reportes.

En la estructura de zonas y asientos se implementó el patrón **Composite**, ya que permite representar relaciones jerárquicas parte-todo, donde una zona puede contener múltiples asientos y ser tratada de manera uniforme dentro del sistema.

El patrón **Decorator** se utilizó en la gestión de compras, permitiendo agregar servicios adicionales como VIP, parqueadero o merchandising de forma dinámica sin modificar la estructura base de la compra.

Para la generación de reportes se implementó el patrón **Adapter**, el cual permite integrar una librería externa de generación de archivos PDF y CSV adaptándola a la estructura interna del sistema.

Por otro lado, el patrón **Proxy** se implementó en el módulo de compras, actuando como intermediario para validar el acceso del usuario al sistema antes de permitir la ejecución de operaciones, garantizando que solo usuarios autenticados puedan realizar acciones sensibles.

El manejo de métodos de pago en las compras se resolvió mediante el patrón **Strategy**, permitiendo aplicar diferentes formas de pago según el contexto del usuario sin afectar la lógica principal de la compra.

Por otro lado, el patrón **Observer** se utilizó para la gestión de incidencias y notificaciones, permitiendo informar automáticamente a diferentes partes del sistema cuando ocurren eventos relevantes.

Finalmente, el patrón **State** se implementó tanto en la gestión de asientos como en la gestión de compras, permitiendo controlar su comportamiento según su estado actual (disponible, reservado, vendido, cancelada, pagada, entre otros), facilitando la transición entre estados sin modificar la lógica principal.

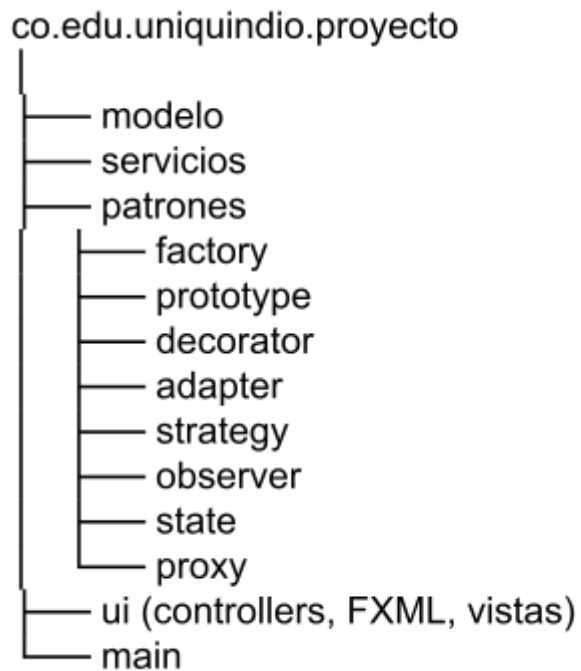
## 7. ¿Cómo se escribe la solución en Java?

La solución se implementa en Java bajo una arquitectura modular basada en capas, con el objetivo de separar responsabilidades y facilitar el mantenimiento del sistema.

El proyecto se organiza en paquetes principales que agrupan las diferentes partes del sistema:

- **modelo:** contiene las entidades del dominio como Usuario, Evento, Compra, Entrada, Recinto, Zona, Asiento, Pago, Tarifa e Incidencia.
- **servicios:** contiene la lógica de negocio del sistema, incluyendo clases como UsuarioService, EventoService, CompraService, IncidenciaService y ReporteService.
- **patrones:** contiene la implementación de los patrones de diseño utilizados, organizados en subpaquetes como factory, builder/prototype, decorator, adapter, strategy, observer, state y proxy.
- **ui:** contiene la interfaz gráfica desarrollada en JavaFX.
- **main:** contiene la clase principal desde donde se ejecuta la aplicación.

Cada funcionalidad del sistema se implementa a través de métodos en la capa de servicios, los cuales interactúan con las entidades del modelo y aplican los patrones de diseño según sea necesario.



**Enlace al diagrama de clases:**

[https://miro.com/app/board/uXjVHV98fg0=/?share\\_link\\_id=839295507157](https://miro.com/app/board/uXjVHV98fg0=/?share_link_id=839295507157)